

MINISTRY OF EDUCATION, CULTURE AND RESEARCH
OF THE REPUBLIC OF MOLDOVA

Technical University of Moldova

Faculty of Computers, Informatics and Microelectronics

Department of Software Engineering and Automatics

Mihail Pancenco FAF-243

Report

Laboratory work n. 4
of Operating Systems

The xv6 Shell from the Outside

Supervisers:

Dumitrașcu Marius, *University assistant*

Reițman Patricia, *University assistant*

DISA, FCIM, UTM

Chișinău – 2026

Table of Contents

1 Introduction **3**

2 Tasks **4**

2.1 Fix sleep for the current xv6 4

2.2 Bash versus the xv6 shell 5

2.2.1 Shell comparison 5

2.2.2 Features supported by both shells 7

2.2.3 Features available only in Bash 8

2.2.4 Executing programs after changing directories 9

2.2.5 Evaluation of missing features 10

2.3 Who expands *? A program that prints its argv 11

2.3.1 Implementation for xv6 11

2.3.2 Implementation for Linux 11

2.3.3 Comparison of command-line arguments 11

2.3.4 Argument expansion and quote processing 11

2.3.5 Whitespace handling 11

2.4 Read the shell 11

2.4.1 Source code inspection 11

2.4.2 Command types supported by xv6 11

2.4.3 Implementation of cd 11

2.4.4 Missing variable and wildcard expansion 11

3 CONCLUSIONS **12**

1 Introduction

2 Tasks

2.1 Fix sleep for the current xv6

The first task was to ensure that the `sleep` utility was compatible with the current version of xv6.

In August 2025, the xv6 system call `sleep()` was renamed to `pause()`. This change had already been addressed during Laboratory Work No. 2, so no additional modifications were required.

The existing implementation in `user/sleep.c` uses the updated system call:

```
pause(atoi(argv[1]));
```

The program also validates the number of command-line arguments. If the user does not provide exactly one argument, it displays a usage message and terminates with a nonzero exit status.

To verify the implementation, the xv6 system was started using:

```
make qemu
```

Inside the xv6 shell, the following commands were executed:

```
$ sleep 20
$ sleep
usage: sleep <ticks>
```

The first command successfully paused execution for 20 timer ticks and returned to the shell. The second command, executed without the required argument, correctly displayed the usage message.

Figure 2.1 shows the existing source code, the successful xv6 boot process, and the results of both tests.

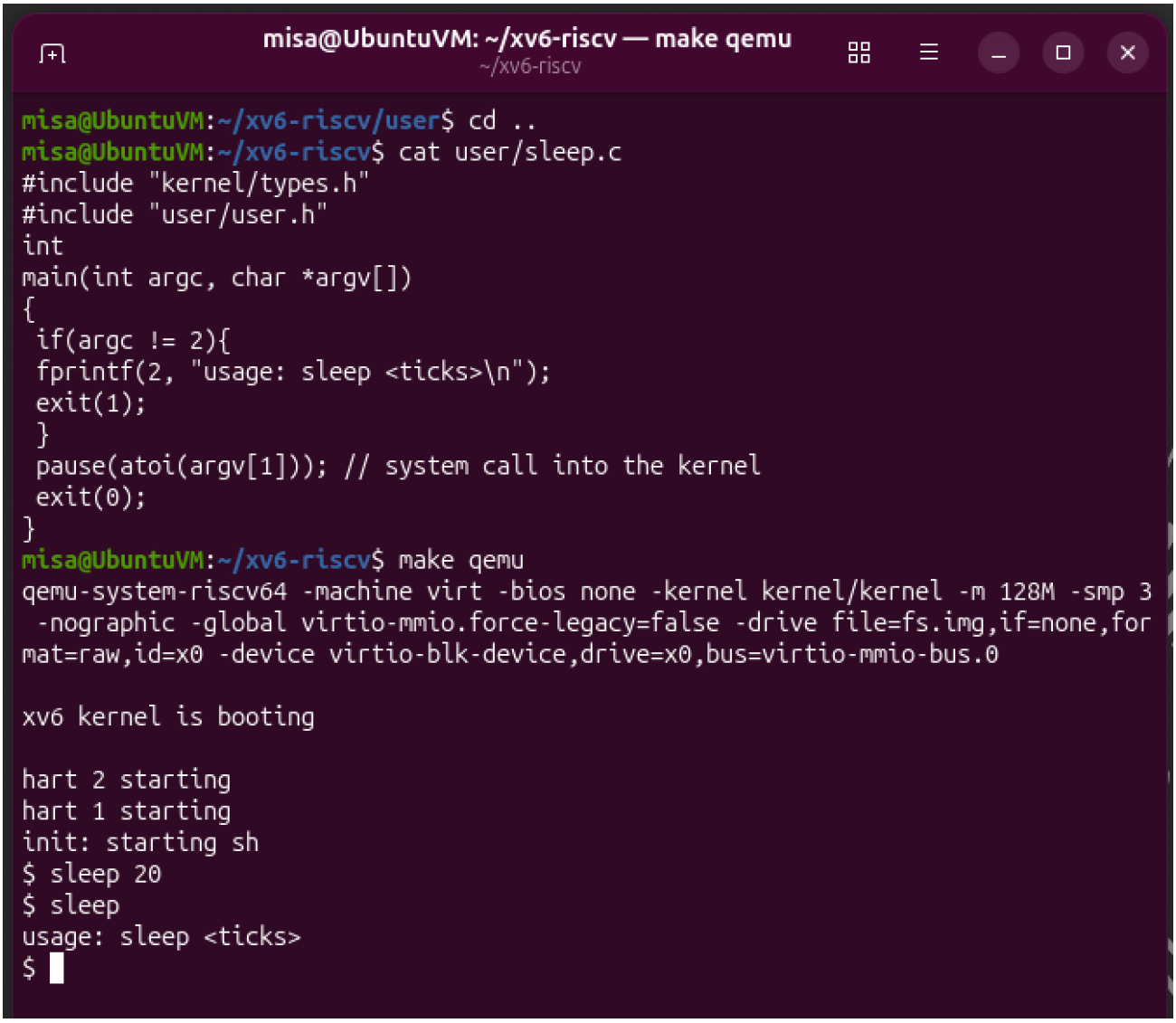


Figure 2.1 - Verification of the sleep utility using the pause system call in xv6

The results confirm that the `sleep` utility from Laboratory Work No. 2 remains compatible with the current xv6 version. Both normal execution and argument validation work as expected.

2.2 Bash versus the xv6 shell

The second task was to compare the behavior of Bash, the standard shell used in Ubuntu, with the minimal shell provided by xv6.

The same commands were executed in both environments to investigate redirection, pipelines, command grouping, sequential execution, background processes, wildcard expansion, environment variables, quoting, directory navigation, and interactive shell features.

For the Bash experiments, a test directory was created in Ubuntu. The xv6 experiments were performed directly in the shell running inside QEMU.

2.2.1 Shell comparison

Table 2.1 summarizes the observed results.

Command	Tests	Bash	xv6
`echo hello > f ; cat < f`	Redirection	Prints `hello`.	Prints `open f failed`.
`ls | grep READ`	Pipe	No matching output.	No matching output.
`(echo a; echo b) | wc`	Grouping, pipe	Prints `2 2 4`.	Prints `2 2 4`.
`echo one ; echo two`	Sequence	Prints `one` and `two`.	Prints `one` and `two`.
`echo bg &`	Background	Prints a job number, PID, and `bg`.	Prints `bg`; the prompt may appear immediately.
`ls *`	Globbing	Expands `*`; prints `f`.	Prints `ls: cannot open *`.
`echo $HOME`	Variables	Prints `/home/misa`.	Prints the literal `$HOME`.
`echo "a b"`	Quoting	Prints `a b`.	Prints `"a b"`, including quotation marks.
`mkdir d ; cd d ; ls ..`	Directory navigation	Changes into `d` and lists `d` and `f`.	Prints `exec cd failed`; lists the root directory.
Arrow Up, Tab, Ctrl+R	Interactive features	History navigation, completion, and reverse search work.	No corresponding interactive features were observed.

Table 2.1 - Experimental comparison of Bash and the xv6 shell

Figure 2.2 shows the commands executed in both environments and their corresponding outputs.

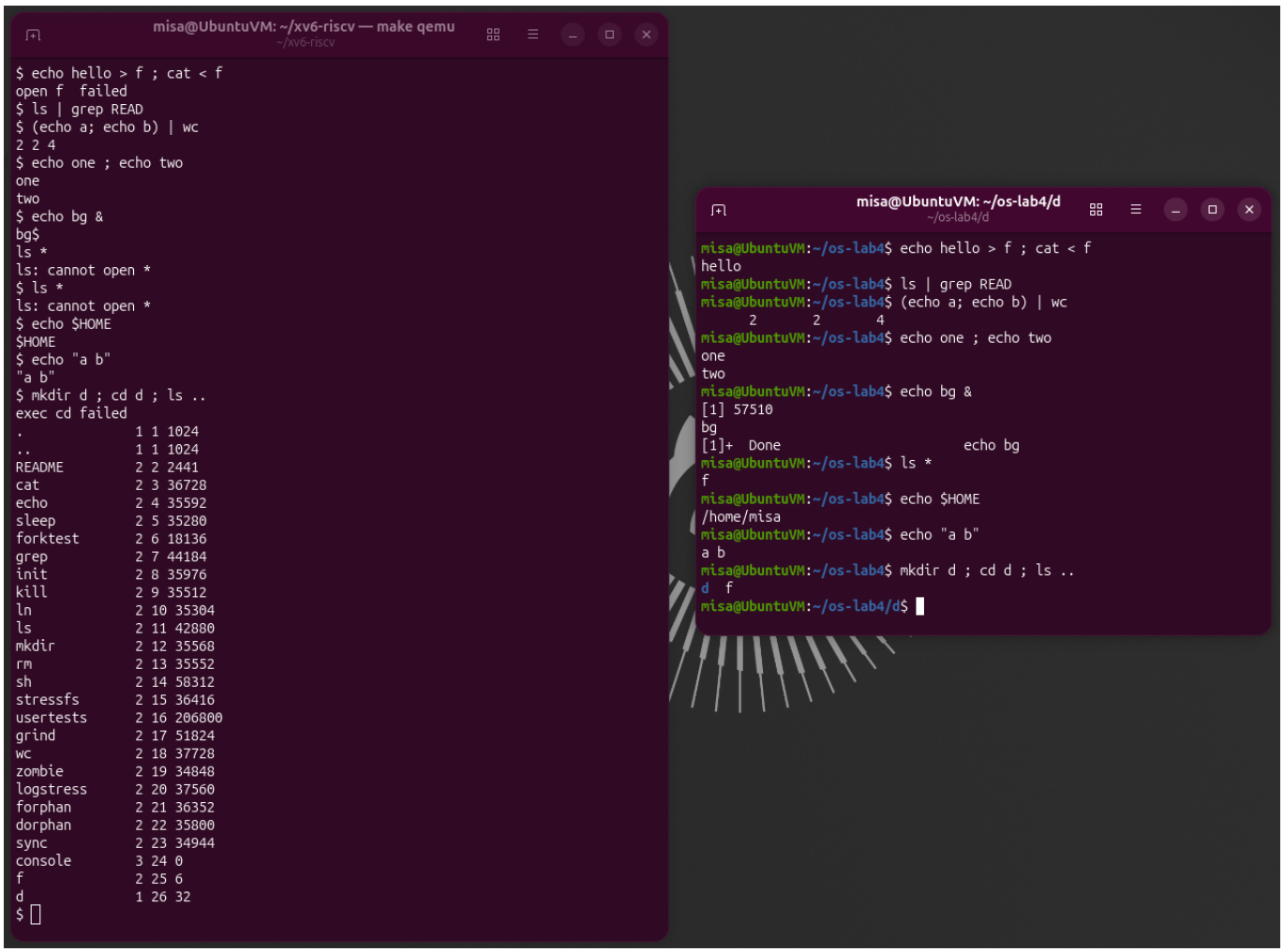


Figure 2.2 - Comparison of command execution in the xv6 shell (left) and Ubuntu Bash (right)

2.2.2 Features supported by both shells

The experiments demonstrated that both shells support several fundamental mechanisms required for command execution.

- **Input and output redirection.** Both shells recognize redirection operators such as > and <. However, the combined redirection and command sequence produced an error in xv6, which is discussed below.
- **Pipelines.** Both shells support the pipe operator (`|`), which connects the standard output of one command to the standard input of another.
- **Command grouping.** Parentheses can be used to group commands. The command `(echo a; echo b) | wc` produced the same result in both environments: two lines, two words, and four bytes.
- **Sequential execution.** The semicolon (`;`) allows multiple commands to be executed sequentially. Both shells successfully executed `echo one ; echo two`.
- **Background execution.** The ampersand (`&`) allows a command to execute without waiting for its completion. Bash additionally displayed job-control information, while xv6 provided only minimal output.

The first experiment revealed an important difference in how the two shells process compound commands:

```
echo hello > f ; cat < f
```

In Bash, the command created the file `f`, wrote `hello` into it, and then read the file successfully.

In xv6, the same command produced:

```
open f failed
```

This result does not indicate that xv6 lacks redirection. Instead, it reflects the interaction between redirection and the command sequence in this particular experiment.

The xv6 shell executes the commands separated by `;` using its own process-management logic. Unlike Bash, it does not necessarily wait for the first command to complete before starting the second. Consequently, `cat` may attempt to open `f` before the preceding command has created it.

2.2.3 Features available only in Bash

Several differences demonstrate that Bash provides more advanced command-line processing than the xv6 shell.

Wildcard expansion

The command

```
ls *
```

produced different results.

In Bash, the wildcard `*` was expanded into matching filenames before the command was executed. In the test directory, this resulted in the filename `f` being passed to `ls`.

In xv6, the wildcard was not expanded. Instead, `ls` received the literal character `*` as an argument and attempted to open a file with that name:

```
ls: cannot open *
```

This demonstrates that wildcard expansion is a feature of the shell rather than the `ls` program itself.

Environment variable expansion

The command

```
echo $HOME
```

printed the user's home directory in Bash:

```
/home/misa
```

In xv6, the same command printed:

```
$HOME
```

Bash interpreted `$HOME` as a variable reference and substituted its value. The xv6 shell did not perform this substitution and passed the text directly to `echo`.

Quotation handling

The command

```
echo "a b"
```

produced the following output in Bash:

```
a b
```

In xv6, the output was:

```
"a b"
```

Bash interpreted the quotation marks as shell syntax and removed them before executing the command. The xv6 shell preserved the quotation marks as ordinary characters.

This experiment shows that the two shells process command-line arguments differently.

Command history and completion

The following keyboard shortcuts were tested:

- **Arrow Up:** Bash retrieved previously executed commands. In xv6, the key did not retrieve command history.
- **Tab:** Bash performed command and filename completion. In xv6, the key inserted a tab character instead.
- **Ctrl+R:** Bash activated reverse incremental history search. In xv6, no corresponding search functionality was observed.

These differences indicate that the xv6 shell provides only a minimal interactive interface, without the convenience features commonly available in Bash.

2.2.4 Executing programs after changing directories

The following command was executed in both environments:

```
mkdir d ; cd d ; ls ..
```

In Bash, the directory `d` was created successfully, the shell changed into it, and `ls ..` displayed the contents of the parent directory:

```
d  f
```

In xv6, the output began with:

```
exec cd failed
```

The subsequent `ls ..` command displayed the contents of the xv6 root directory, indicating that the shell had not changed its working directory.

This behavior is caused by the way the xv6 shell handles the `cd` command.

Unlike Bash, which supports command lists containing built-in operations, the xv6 shell handles `cd` specially when it is entered as a standalone command.

When `cd d` appears inside a command sequence, it is processed as an ordinary executable command. Since no separate `cd` executable exists, the shell reports `exec cd failed`.

Even when `cd` is executed successfully as a standalone command, another limitation remains: xv6 does not provide Bash-style `PATH` lookup for external programs.

The xv6 user programs are stored in the root directory. After changing into a subdirectory, a command such as

```
ls
```

may fail because the executable is no longer located in the current directory.

The program can still be executed by specifying its absolute path:

```
/ls
```

Alternatively, the relative path to the parent directory can be used:

```
../ls
```

This demonstrates that the xv6 shell relies on explicit executable paths rather than searching directories listed in an environment variable.

2.2.5 Evaluation of missing features

The most important missing feature for everyday use is executable search through a mechanism similar to Bash's `PATH` variable.

In xv6, executable programs are stored in the root directory. After changing to another directory, commands such as `ls`, `cat`, or `grep` cannot be executed by name alone. The user must specify an explicit path, such as `/ls`.

A PATH-like mechanism would eliminate this limitation and make command execution independent of the current working directory.

This feature would also be relatively straightforward to implement in a simplified form. For example, the shell could attempt to execute a command normally and, if that fails, search for the executable in a predefined directory such as the root directory.

Other useful features include command history, Tab completion, and wildcard expansion. However, these primarily improve convenience, whereas executable search addresses a more fundamental usability limitation.

Therefore, executable path lookup would be my first priority when extending the xv6 shell.

2.3 Who expands *? A program that prints its argv

2.3.1 Implementation for xv6

2.3.2 Implementation for Linux

2.3.3 Comparison of command-line arguments

2.3.4 Argument expansion and quote processing

2.3.5 Whitespace handling

2.4 Read the shell

2.4.1 Source code inspection

2.4.2 Command types supported by xv6

2.4.3 Implementation of cd

2.4.4 Missing variable and wildcard expansion

3 CONCLUSIONS